

Sparse Curriculum Sampling for Efficient Fine-Tuning of Small Language Models

An Tran and Laura Hayes
Demo Institute of Computing (fictional affiliation)

Abstract

Fine-tuning a pretrained language model on a new task usually requires many passes over the training set, even though a large fraction of the examples are already handled well after the first epoch. We propose Sparse Curriculum Sampling (SCS), a simple data selection method that estimates the difficulty of each example from its recent loss and samples hard examples more often while still revisiting easy ones. SCS adds no trainable parameters and can be implemented in a few lines of code. On three text classification benchmarks, SCS reaches the accuracy of standard fine-tuning while using 38% fewer gradient updates, and it reduces the variance across random seeds. We also discuss when the method fails and why its gains may not transfer to generation tasks.

1 Introduction

Small language models with fewer than one billion parameters remain attractive for laboratories and companies with limited computing budgets. However, fine-tuning such a model still involves a trade-off between cost and accuracy: training for more epochs tends to improve the final score, but most of the additional computation is spent on examples that the model has already learned.

Curriculum learning [2] addresses a related problem by presenting examples in a meaningful order, usually from easy to hard. In practice, a fixed curriculum requires a difficulty measure defined before training, which is often unavailable for new datasets. In this paper, we take a different view: instead of ordering the data once, we let the model's own training signal decide how often each example should be seen.

Our contributions are threefold. First, we introduce SCS, a sampling rule based on an exponential moving average of the per-example loss. Second, we show empirically that SCS reduces the number of gradient updates without hurting accuracy. Third, we analyze the failure cases of the method, which, to the best of our knowledge, have not been reported for similar approaches.

2 Method

Let $D = \{(x_i, y_i)\}$, $i = 1..N$, be the training set and $f(x; w)$ a classifier with weights w . At step t , we keep a difficulty score s_i for every example, updated whenever the example is visited:

$$s_i \leftarrow \beta * s_i + (1 - \beta) * L(f(x_i; w), y_i) \quad (1)$$

where L is the cross-entropy loss and β is a smoothing factor set to 0.9 in all experiments. Examples are then drawn with probability

$$p_i = (s_i + \epsilon)^\alpha / \sum_j (s_j + \epsilon)^\alpha \quad (2)$$

The exponent α controls how strongly the sampler focuses on hard examples: $\alpha = 0$ recovers uniform sampling, whereas large values concentrate the probability mass on a small

** This is a fictional paper written for the PaperViet demo. The method and all numbers are invented; only the references are real.*

subset. A small constant eps guarantees that every example keeps a non-zero probability of being selected, so that easy examples are revisited and the model does not forget them. We optimize the weights with Adam [4] and apply dropout [3] with rate 0.1, as in standard fine-tuning of Transformer encoders [1].

3 Experiments

We fine-tune a 110M-parameter encoder on three public text classification datasets, denoted A, B and C, with 12k, 45k and 120k training examples. Each configuration is repeated with five random seeds, and we report the mean accuracy on the test set together with the standard deviation. The baseline uses uniform sampling with the same learning rate of $2e-5$ and a batch size of 32.

Table 1: Test accuracy (%) and number of gradient updates (thousands) on the three datasets. Values are mean $\pm$ standard deviation over five seeds.

Method	Data A	Data B	Data C	Updates (k)
Uniform	86.1 $\pm$ 0.9	90.4 $\pm$ 0.5	93.2 $\pm$ 0.3	52.0
SCS (alpha = 1)	86.3 $\pm$ 0.6	90.6 $\pm$ 0.4	93.1 $\pm$ 0.2	32.2
SCS (alpha = 2)	85.2 $\pm$ 1.1	89.9 $\pm$ 0.7	92.8 $\pm$ 0.4	30.5

As shown in Table 1, SCS with alpha = 1 matches or slightly exceeds the baseline on two of the three datasets and stays within 0.1 points on the third, while requiring 38% fewer updates (32.2k versus 52.0k). The lower standard deviation suggests that focusing on informative examples also makes training more stable. With alpha = 2, however, accuracy drops on every dataset, which indicates that an overly aggressive sampler ignores easy examples for too long.

4 Limitations

Our study has several limitations. All experiments use classification tasks and a single model size, so it is unclear whether the gains hold for text generation or for models with billions of parameters. In addition, noisy labels may receive persistently high losses; in such cases SCS could repeatedly sample mislabeled examples and amplify the noise.

5 Conclusion

We presented Sparse Curriculum Sampling, a lightweight method that reuses the training loss to decide which examples deserve more attention. The method reduces the cost of fine-tuning in our setting and is easy to combine with existing training pipelines. Future work will study adaptive values of alpha and robustness to label noise.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [2] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, 2009.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929-1958, 2014.
- [4] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.